

ЦОИ — ЛРЗ — Вейвлет-анализ — вариант 5

По методичке: 3.1 — НВП: фазовая манипуляция, [0 ... 4] мс, $dt = 1e-6$ с 3.2 — ДВП (Haar): тот же $s(t)$, что в таблице вар. 5, сетка 0:1:1000 с 3.3 — wden и ручное обнуление деталей; для вар. 5 реком. СКО шума 0.3

```
clear; close all; clc;

thisDir = fileparts(mfilename('fullpath'));
outDir = fullfile(thisDir, 'Результаты');
if ~exist(outDir, 'dir'); mkdir(outDir); end

% Общее оформление мозаики подписей графиков
tlCmp = {'Padding', 'compact', 'TileSpacing', 'compact'};
```

§3.1 — сигнал для НВП (фазовая манипуляция)

```
dt_cwt = 1e-6;
t_cwt = 0:dt_cwt:0.004;
f0_hz = 10e3;

s31 = zeros(size(t_cwt));
phaseA = (t_cwt >= 0) & (t_cwt <= 0.002);
phaseB = t_cwt > 0.002;
s31(phaseA) = sin(2 * pi * f0_hz .* t_cwt(phaseA));
s31(phaseB) = sin(2 * pi * f0_hz .* t_cwt(phaseB) + pi / 2);
```

§3.2 и §3.3 — сигнал для ДВП / денойзинга

```
t_dwt = 0:1:1000;
[f1_hz, f2_hz, f3_hz] = deal(1e-3, 4e-3, 2e-3);
s_clean = ...
    2 .* sin(2 * pi .* f1_hz .* t_dwt + pi / 2) + ...
    cos(2 * pi .* f2_hz .* t_dwt) + ...
    0.5 .* sin(2 * pi .* f3_hz .* t_dwt);
```

3.1 Непрерывное вейвлет-преобразование

```
scales = 1:128;
wNames = ['mexh', 'haar', 'db4', 'morl'];

fig = figure('Color', 'white', 'Name', '§3.1 — сигнал');
plot(t_cwt, s31, 'k'); grid on;
title('Исходный сигнал (фазовая манипуляция, {\it f} = 10 кГц)');
xlabel('t, c'); ylabel('s(t)');
savePng(fig, outDir, '01_signal_31.png');

fig = figure('Color', 'white', 'Name', 'Материнские вейвлеты');
tiledlayout(fig, 2, 2, tlCmp{:});
```

```

nexttile;
[psi_mex, xm] = mexihat(-5, 5, 1000);
plot(xm, psi_mex, 'k'); grid on;
title('mexh (mexihat)'); xlabel('t'); ylabel('\psi(t)');

nexttile;
[~, psi, xv] = wavefun('haar', 6);
plot(xv, psi, 'k'); grid on;
title('haar (wavefun)'); xlabel('t'); ylabel('\psi(t)');

nexttile;
[~, psi, xv] = wavefun('db4', 6);
plot(xv, psi, 'k'); grid on;
title('db4 (wavefun)'); xlabel('t'); ylabel('\psi(t)');

nexttile;
text(0.06, 0.55, 'mor1 – удобнее смотреть CWT-спектр', 'FontSize', 11);
axis off;

savePng(fig, outDir, '02_mother_wavelets.png');

for kw = 1:numel(wNames)
    wn = wNames(kw);
    fig = figure('Color', 'white', 'Name', ['CWT – ' char(wn)]);
    tiledlayout(fig, 2, 1, t1Cmp{:});

    nexttile;
    plot(t_cwt, s31, 'k'); grid on;
    title('s(t), §3.1'); xlabel('t, c'); ylabel('s(t)');

    nexttile;
    cwt(s31, scales, wn, 'absglb');
    title(['CWT – ' char(wn)]);

    savePng(fig, outDir, ['03_cwt_' char(wn) '.png']);
end

```

3.2 Дискретное вейвлет-преобразование (Haar)

```

wnameHaar = 'haar';
nLev = 3;

[C, L] = wavedec(s_clean, nLev, wnameHaar);

fig = figure('Color', 'white', 'Name', '§3.2 – s и C');
tiledlayout(fig, 2, 1, t1Cmp{:});
nexttile;
plot(t_dwt, s_clean, 'k'); grid on;
title('s(t), §3.2'); xlabel('t, c'); ylabel('s(t)');

```

```

nexttile;
plot(C, 'k'); grid on;
title('Вектор коэффициентов C'); xlabel('индекс'); ylabel('C');
savePng(fig, outDir, '04_dwt_C.png');

Alev = appcoef(C, L, wnameHaar, nLev);
fig = figure('Color', 'white', 'Name', '$3.2 - appcoef / detcoef');
tiledlayout(fig, 2, 2, tlcmp{:});

nexttile;
stem(0:numel(Alev) - 1, Alev, 'filled', 'MarkerSize', 3);
grid on;
title(sprintf('Аппроксимация - уровень %d', nLev)); xlabel('n'); ylabel('A');

for j = 1:nLev
    nexttile;
    Dj = detcoef(C, L, j);
    stem(0:numel(Dj) - 1, Dj, 'filled', 'MarkerSize', 3);
    grid on;
    title(sprintf('Деталь D%d', j)); xlabel('n'); ylabel('D');
end
savePng(fig, outDir, '04_dwt_coeffs.png');

App = cell(nLev, 1); Det = cell(nLev, 1);
for j = 1:nLev
    App{j} = wrcoef('a', C, L, wnameHaar, j);
    Det{j} = wrcoef('d', C, L, wnameHaar, j);
end

clrD = [0.85 0.2 0.2];
fig = figure('Color', 'white', 'Name', '$3.2 - wrcoef по времени');
tiledlayout(fig, 2, 2, tlcmp{:});

nexttile;
plot(t_dwt, s_clean, 'k'); grid on;
title('s(t)'); xlabel('t, c'); ylabel('s');

nexttile;
plot(t_dwt, App{nLev}, 'b'); grid on;
title(sprintf('Аппроксимация A%d', nLev)); xlabel('t, c'); ylabel('A');

nexttile;
plot(t_dwt, Det{1}, 'Color', clrD); grid on;
title('Деталь D1'); xlabel('t, c'); ylabel('D1');

nexttile;
plot(t_dwt, Det{nLev}, 'Color', clrD); grid on;
title(sprintf('Деталь D%d', nLev)); xlabel('t, c'); ylabel(sprintf('D%d', nLev));

savePng(fig, outDir, '05_dwt_app_det.png');

```

Реконструкция по частям

```
% Только аппроксимация
s_A = wrcoef('a', C, L, wnameHaar, nLev);

% Только детали разных уровней
s_D1 = wrcoef('d', C, L, wnameHaar, 1);
s_D2 = wrcoef('d', C, L, wnameHaar, 2);
s_D3 = wrcoef('d', C, L, wnameHaar, 3);

% Комбинации
s_Dsum = s_D1 + s_D2 + s_D3;
s_A_plus_D1 = s_A + s_D1;

% Графики
fig = figure('Color', 'white', 'Name', 'Реконструкция сигналов');
tiledlayout(3,2);

nexttile; plot(t_dwt, s_clean); grid on; title('Исходный сигнал');

nexttile; plot(t_dwt, s_A); grid on;
title('Только аппроксимация A');

nexttile; plot(t_dwt, s_D1); grid on;
title('Только D1 (высокие частоты)');

nexttile; plot(t_dwt, s_Dsum); grid on;
title('Сумма всех деталей');

nexttile; plot(t_dwt, s_A_plus_D1); grid on;
title('A + D1');

savePng(fig, outDir, '05_reconstruction.png');
```

3.3 Шумоподавление (wden) и ручная фильтрация

```
sigma_noise = 0.3;
s_noisy = s_clean + sigma_noise .* randn(size(s_clean));

tptrList = ["sqtwolog", "heursure", "rigrsure", "minimaxi"];
sorthList = ["h", "s"];
scallList = ["one", "sln", "mln"];
waveList = ["db4", "db8", "sym4"];
levList = [3, 4, 5];

idx = 0;

for w = waveList
    for lv = levList
```

```

for tp = tptrList
    for sh = sorhList
        for sc = scalList
            idx = idx + 1;
            try
                s_hat = wden(s_noisy, tp, sh, sc, lv, w);
                mse = mean((s_clean - s_hat) .^ 2);
                ok = true;
            catch
                mse = NaN;
                ok = false;
            end
            res(idx).wave = w;
            res(idx).lev = lv;
            res(idx).tptr = tp;
            res(idx).sorh = sh;
            res(idx).scal = sc;
            res(idx).mse = mse;
            res(idx).ok = ok;
        end
    end
end
end

T = struct2table(res);
T = sortrows(T, 'mse', 'ascend', 'MissingPlacement', 'last');
writetable(T, fullfile(outDir, '06_wden_mse_table.csv'));
save(fullfile(outDir, '06_wden_mse_table.mat'), 'T');

T_ok = T(T.ok == 1 & ~isnan(T.mse), :);
best = T_ok(1, :);
s_wden = wden(s_noisy, best.tptr{1}, best.sorh{1}, ...
    best.scal{1}, best.lev(1), best.wave{1});

```

3.4 Ручная фильтрация (частичное удаление деталей)

```

wnameMan = 'db8';
levMan = 5;

[Cm, Lm] = wavedec(s_noisy, levMan, wnameMan);

levelsToRemove = [1 2]; % удаляем только высокочастотные

Cm2 = Cm;

for k = levelsToRemove
    Cm2 = wthcoef('d', Cm2, Lm, k);
end

```

```

s_manual = waverec(Cm2, Lm, wnameMan);

mseManual = mean((s_clean - s_manual).^2);

mseWden = mean((s_clean - s_wden) .^ 2);
mseMan = mean((s_clean - s_manual) .^ 2);

fig = figure('Color', 'white', 'Name', '$3.3 – сравнение');
tiledlayout(fig, 3, 1, tlCmp{:});

nexttile;
plot(t_dwt, s_clean, 'k'); grid on;
title('Чистый сигнал'); ylabel('s');

nexttile;
plot(t_dwt, s_noisy, 'Color', [0.35 0.35 0.35]); grid on;
title(sprintf('Зашумление (sigma = %.1f)', sigma_noise)); ylabel('s_{noisy}');

nexttile;
plot(t_dwt, s_wden, 'b'); hold on;
plot(t_dwt, s_manual, 'r'); hold off; grid on;
legend('wden', 'manual', 'Location', 'best');
title(sprintf(['После фильтрации – MSE: ', ...
    'wden=%.4g, manual=%.4g'], mseWden, mseMan));
xlabel('t, c'); ylabel('estimate');

savePng(fig, outDir, '07_denoising_compare.png');

save(fullfile(outDir, '07_signals.mat'), ...
    't_cwt', 's31', 't_dwt', 's_clean', ...
    's_noisy', 's_wden', 's_manual', 'best', 'sigma_noise');

```

Подбор уровней удаления

```

results_manual = [];

for maxLevelRemove = 1:4
    Cm2 = Cm;

    for k = 1:maxLevelRemove
        Cm2 = wthcoef('d', Cm2, Lm, k);
    end

    s_tmp = waverec(Cm2, Lm, wnameMan);
    mse_tmp = mean((s_clean - s_tmp).^2);

    results_manual = [results_manual; maxLevelRemove mse_tmp];
end

disp('Удаление уровней / MSE:');

```

Удаление уровней / MSE:

```
disp(results_manual);
```

1.0000	0.0421
2.0000	0.0217
3.0000	0.0115
4.0000	0.0055

```
fprintf('Готово: %s\n', outDir);
```

Готово: C:\Users\artem\AppData\Local\Temp\Editor_ruzou\Результаты

локальные функции

```
function savePng(fig, outDir, fname)
exportgraphics(fig, fullfile(outDir, fname), 'Resolution', 200);
close(fig);
end
```